

Testes Integrados

Os testes integrados são essenciais para garantir a qualidade e a funcionalidade de aplicações complexas. Eles verificam a interação entre diferentes módulos ou componentes do sistema, assegurando que trabalham juntos como esperado. Este texto explora o conceito, os benefícios, as práticas recomendadas e as ferramentas utilizadas para realizar testes integrados no desenvolvimento de apps.



Fonte: <https://pixabay.com/pt/vectors/graduação-diplomado-aluna-feliz-149646/>

O que são Testes Integrados?

Os testes integrados avaliam a comunicação entre diferentes partes de um sistema. Em vez de testar componentes de forma isolada, eles simulam cenários reais para verificar se as partes do sistema funcionam corretamente quando combinadas.

Por que Realizar Testes Integrados?

- **Detecção de Problemas na Comunicação:** Identifica falhas na troca de dados entre módulos.
- **Validação de Fluxos de Trabalho:** Garante que os processos de ponta a ponta funcionem como planejado.
- **Prevenção de Regressões:** Evita que novas alterações no código quebrem funcionalidades existentes.

Ferramentas Populares para Testes Integrados

- **Flutter Integration Tests:** Ferramenta nativa para criar testes em apps Flutter.
- **Firebase Test Lab:** Permite testar apps em dispositivos reais hospedados na nuvem.
- **Appium:** Ferramenta de código aberto para automação de testes em múltiplas plataformas.
- **JUnit:** Útil para testes de integração em backend e APIs.

Passos para Realizar Testes Integrados

- **Planejamento:** Identificar os cenários de integração que precisam ser validados.
- **Configuração do Ambiente:** Garantir que os módulos e sistemas estejam disponíveis para testes.
- **Criação dos Testes:** Escrever scripts ou casos de teste para cobrir os cenários identificados.
- **Execução:** Rodar os testes e monitorar os resultados.
- **Correção e Reexecução:** Resolver problemas detectados e repetir os testes para validação.

Boas Práticas

- **Automatize Sempre que Possível:** Reduza esforço humano e aumente a consistência.
- **Defina Cobertura de Testes:** Garanta que todos os fluxos principais sejam testados.
- **Integre Testes ao CI/CD:** Execute testes automaticamente em cada alteração de código.
- **Documente Erros e Soluções:** Facilite a resolução de problemas recorrentes.

GitHub da Disciplina

Você pode acessar o repositório da disciplina no GitHub a partir do seguinte link:

<https://github.com/FaculdadeDescomplica/Pratica-Integradora-Desenvolvimento-de-Apps>. Esse espaço é o seu portal para mergulhar fundo no universo da aprendizagem interativa. Nele, você encontrará todos os códigos, além dos links para os arquivos e dados.

Conteúdo Bônus

Saiba mais sobre como implementar testes integrados com Flutter:

Título: Check app functionality with an integration test

Plataforma: Flutter

Referências Bibliográficas

BIESSEK, Alessandro. Flutter for Beginners. 1. ed. Birmingham: Packt Publishing, 2020.

SEJOURNÉ, Philippe. Flutter: Aplicações Nativas Multiplataforma com Flutter e Dart. 1. ed. Rio de Janeiro: Alta Books, 2020.

ZAMMETTI, Frank. Practical Flutter: Improve your Mobile Development with Google's Latest Open-Source Framework. 1. ed. Berkeley: Apress, 2019.

Atividade Prática 15 - Testes Integrados

Título da Prática: Realizando Testes Integrados em um Aplicativo Flutter

Objetivos: Compreender como realizar testes integrados em um aplicativo móvel, verificando a integração entre diferentes componentes, como interfaces de usuário, banco de dados e APIs externas.

Materiais, Métodos e Ferramentas: Para esta prática, utilize o Flutter com o Flutter Test para realizar testes integrados. O Android Studio será a ferramenta recomendada para execução e análise.

Roteiro

1º Passo) Leia atentamente o texto.

Os testes integrados são fundamentais para garantir que diferentes componentes do aplicativo funcionem corretamente quando interagem entre si. No desenvolvimento de aplicativos móveis, esses testes podem incluir a interação entre a interface gráfica (UI), APIs externas, banco de dados local, e outros serviços do aplicativo.

No Flutter, podemos realizar testes integrados utilizando a biblioteca **flutter_test** para verificar se diferentes partes do aplicativo estão funcionando de forma coesa. O objetivo é testar o comportamento de telas, a manipulação de dados e a comunicação com APIs.

2º Passo) Configuração do projeto para testes integrados.

1. Crie um novo projeto no **Flutter**.
2. No arquivo pubspec.yaml, adicione a dependência para testes:

yaml

CopiarEditar

dev_dependencies:

flutter_test:

sdk: flutter

Se estiver testando API

http: ^0.13.3

3. Crie uma pasta chamada test no seu projeto para organizar os testes.

3º Passo) Criando um teste integrado simples.

Vamos simular um teste integrado para um aplicativo que exibe uma lista de usuários obtidos de uma API externa.

Para isso, vamos realizar a integração com uma **API fictícia** (por exemplo, JSONPlaceholder) para buscar dados de usuários e exibi-los em uma lista. O teste será realizado para garantir que a tela de exibição dos usuários esteja funcionando corretamente.

Aqui está o código do aplicativo básico:

dart

CopiarEditar

```
import 'package:flutter/material.dart';
```

```
import 'package:http/http.dart' as http;
```

```
import 'dart:convert';
```

```
void main() {
```

```
  runApp(MyApp());
```

```
}
```

```
class MyApp extends StatelessWidget {
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
return MaterialApp(  
  home: UsersScreen(),  
);  
}  
}
```

```
class UsersScreen extends StatefulWidget {  
  @override  
  _UsersScreenState createState() => _UsersScreenState();  
}
```

```
class _UsersScreenState extends State<UsersScreen> {  
  List<dynamic> _users = [];
```

```
  @override  
  void initState() {  
    super.initState();  
    _fetchUsers();  
  }
```

```
  Future<void> _fetchUsers() async {  
    final response = await  
    http.get(Uri.parse('https://jsonplaceholder.typicode.com/users'));
```

```
    if (response.statusCode == 200) {  
      setState(() {  
        _users = json.decode(response.body);  
      });  
    } else {  
      throw Exception('Failed to load users');  
    }  
  }
```

```
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  

```

```
appBar: AppBar(title: Text('Lista de Usuários')),
body: ListView.builder(
  itemCount: _users.length,
  itemBuilder: (ctx, index) {
    return ListTile(
      title: Text(_users[index]['name']),
      subtitle: Text(_users[index]['email']),
    );
  },
),
);
}
```

4º Passo) Criando o teste integrado.

Agora, vamos criar um teste que simula o processo de buscar os dados e exibir os usuários.

```
dart
CopiarEditar
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/main.dart'; // Substitua pelo caminho correto do seu
app

void main() {
  testWidgets('Deve exibir a lista de usuários', (WidgetTester tester) async {
    // Construir o widget
    await tester.pumpWidget(MyApp());

    // Simular a espera da API
    await tester.pump(Duration(seconds: 2));

    // Verificar se a tela contém o título esperado
    expect(find.text('Lista de Usuários'), findsOneWidget);
  });
}
```

```
// Verificar se a lista de usuários está visível  
expect(find.byType(ListView), findsOneWidget);
```

```
// Verificar se ao menos um item da lista está visível  
expect(find.byType(ListTile), findsWidgets);  
});  
}
```

5º Passo) Executando o teste.

Para rodar o teste, utilize o seguinte comando no terminal dentro do seu projeto Flutter:

```
bash  
CopiarEditar  
flutter test
```

Isso irá rodar o arquivo de teste e exibir os resultados no terminal, permitindo verificar se o teste passou ou falhou.

6º Passo) Responda às questões abaixo:

- A) O que é um teste integrado?
- B) Qual biblioteca é usada no Flutter para realizar testes de widgets?
- C) O que a função `tester.pump()` faz no teste?
- D) Por que a função `testWidgets` é usada?

Gabarito Atividade Prática

A) Um **teste integrado** verifica a **interação entre diferentes componentes** do aplicativo, garantindo que módulos distintos funcionem corretamente juntos.

B) A biblioteca usada no Flutter para realizar testes de widgets é **flutter_test**, pois ela fornece ferramentas para testar a interface gráfica e o comportamento dos widgets.

C) A função **tester.pump()** simula o **tempo de carregamento de um widget**, permitindo que o teste atualize e renderize a interface corretamente.

D) A função **testWidgets** é usada para **testar a interação de widgets e a interface do usuário**, garantindo que os componentes da UI funcionem conforme o esperado.

Comentários e Conclusões

1. O que aprendeu com esta atividade?
2. Como a realização de testes integrados ajuda a melhorar a qualidade do aplicativo?
3. Quais dificuldades você encontrou ao realizar os testes integrados?